

## LAB 2

# Docker Build & Run

### วัตถุประสงค์

- ✓ ฝึก docker build, run, stop, rm
- ✓ ทำความเข้าใจ Dockerfile แต่ละ instruction
- ✓ ทดสอบ FastAPI ที่รันอยู่ใน Container

### สิ่งที่ต้องเตรียม

- Docker Desktop — ตรวจสอบด้วย: `docker --version`
- Repo: <https://github.com/layel2/basic-docker>
- Browser หรือ curl สำหรับทดสอบ API

## ขั้นตอน

### ขั้นตอนที่ 1 Clone Repository

```
git clone https://github.com/layel2/basic-docker.git
cd basic-docker
```

### ขั้นตอนที่ 2 ศึกษา Dockerfile

เปิดไฟล์ Dockerfile และทำความเข้าใจแต่ละ instruction

```
cat Dockerfile
```

ผลลัพธ์ที่จะเห็น:

```
FROM python:3.11-slim
ADD ./requirements.txt /app/requirements.txt
WORKDIR /app
RUN pip install --upgrade pip
RUN pip install -r requirements.txt
ADD ./src/ /app/
EXPOSE 8000
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000", "--reload"]
```

### ขั้นตอนที่ 3 ดู Dependencies

```
cat requirements.txt
```

จะเห็น: fastapi, uvicorn, httpx, pytest

### ขั้นตอนที่ 4 Build Docker Image

-t คือ tag (ชื่อ image) | . คือ Dockerfile อยู่ที่ directory ปัจจุบัน

```
docker build -t my-fastapi-app .
```

 ครั้งแรกอาจใช้เวลา 2-3 นาที | ตรวจสอบ image ที่ build แล้ว:

```
docker images
```

### ขั้นตอนที่ 5 Run Container

-d = detach (background) | -p host:container | --name = ตั้งชื่อ container

```
docker run -d -p 8000:8000 --name my-api my-fastapi-app
docker ps                # ตรวจสอบ container ที่รันอยู่
```

### ขั้นตอนที่ 6 ทดสอบ API

- เปิด browser → <http://localhost:8000>
- ดู Swagger UI → <http://localhost:8000/docs>
- หรือทดสอบผ่าน Terminal:

```
curl http://localhost:8000
curl http://localhost:8000/items
curl http://localhost:8000/health
```

### ขั้นตอนที่ 7 ดู Logs

```
docker logs my-api        # ดู logs ทั้งหมด
docker logs -f my-api     # ดู live logs (Ctrl+C เพื่อออก)
```

### ขั้นตอนที่ 8 รัน Unit Tests ใน Container

รัน pytest โดยตรงใน container ที่กำลังทำงานอยู่

```
docker exec my-api pytest /app/tests/ -v
```

## ขั้นตอนที่ 9 หยุดและลบ Container

```
docker stop my-api  
docker rm my-api  
docker ps -a           # ตรวจสอบว่าลบแล้ว
```

# Docker Compose, Network & Volume

## วัตถุประสงค์

- ✓ ใช้คำสั่ง docker compose up/down กับ multi-container app
- ✓ ทดลอง Docker Network — container คุยหากัน / ไม่คุยหากัน
- ✓ ทดลอง Volume mount local directory และ share ระหว่าง 2 containers

## สิ่งที่ต้องเตรียม

- ผ่าน Lab 2 มาแล้ว
- Docker Desktop (รวม Docker Compose)
- Repo: layel2/basic-docker (ที่ clone ไว้แล้ว)

## ส่วนที่ 1 — Docker Compose

### ขั้นตอนที่ 1 ศึกษา docker-compose.yml

ใน repo basic-docker มีไฟล์ docker-compose.yml อยู่แล้ว ลองดูโครงสร้าง

```
cat docker-compose.yml
```

### ผลลัพธ์ที่จะเห็น:

```
version: '3.8'
services:
  app:
    build: .
    ports:
      - "8000:8000"
    environment:
      - APP_ENV=development
    volumes:
      - ./src:/app          # bind mount - hot reload
    depends_on:
      - redis
  redis:
    image: redis:7-alpine
    ports:
      - "6379:6379"
    volumes:
```

```
- redis_data:/data # named volume - data persist
volumes:
  redis_data:
```

## ขั้นตอนที่ 2 เริ่ม Services และทดสอบ

```
docker compose up -d
docker compose ps # ดูสถานะทุก service

curl http://localhost:8000 # ทดสอบ API
curl http://localhost:8000/items

docker compose exec app env # ดู env vars
# ตรวจสอบว่า APP_ENV=development ปรากฏ
```

## ขั้นตอนที่ 3 ทดสอบ Bind Mount (Hot Reload)

แก้ไขโค้ดใน src/ บน host แล้วดูว่า container เห็นการเปลี่ยนแปลงทันที

```
echo "# hot reload test" >> src/main.py
docker compose exec app tail -3 /app/main.py
# ควรเห็น: # hot reload test
```

## ขั้นตอนที่ 4 Cleanup Compose

```
docker compose down -v # หยุด + ลบ containers, networks, volumes
```

## ขั้นตอนที่ 5 ดูโครงสร้าง lab2b-examples ใน repo

ใน repo basic-docker มีโฟลเดอร์ lab2b-examples/ พร้อม docker-compose.yml สำเร็จรูปสำหรับแต่ละตัวอย่าง

```
cd basic-docker
ls lab2b-examples/
# network-example/ volume-bind/ volume-shared/
```

## ส่วนที่ 2 — Docker Network

ทดสอบว่า containers ที่อยู่ใน user-defined network เดียวกันสามารถ ping หากันได้ด้วย container name แต่ containers ที่ไม่ได้เชื่อม network จะสื่อสารไม่ได้

เปิดดู docker-compose.yml ก่อนรัน: มี server1/server2 ใน my-network และ server3 ที่ไม่ได้เชื่อม

## ขั้นตอนที่ 6 รันและทดสอบ Ping

```

cd lab2b-examples/network-example
cat docker-compose.yml          # ศึกษา compose file ก่อน
docker compose up -d
docker compose ps

# ✅ Ping สำเร็จ — server1 และ server2 อยู่ใน my-network เดียวกัน
docker compose exec server2 ping -c 4 server1
# PING server1: 64 bytes from 172.18.0.2: seq=0 ttl=64
# 4 packets transmitted, 4 packets received, 0% packet loss

# ❌ Ping ล้มเหลว — server3 ไม่ได้อยู่ใน my-network
docker compose exec server3 ping -c 2 server1
# ping: bad address 'server1'

# ตรวจสอบ containers ที่เชื่อมอยู่ใน network
docker network inspect network-example_my-network

docker compose down
cd ../../

```

## ส่วนที่ 3 — Docker Volume

ทดลอง 2 รูปแบบ: (A) Bind Mount — mount local directory เข้า container (B) Named Volume — share ระหว่าง 2 containers

### ขั้นตอนที่ 7 ทดสอบ Bind Mount

เปิดดู volume-bind/docker-compose.yml — mount ./data เข้า /app/data ใน container

```

cd lab2b-examples/volume-bind
cat docker-compose.yml          # ศึกษา compose file
docker compose up -d

# container อ่านไฟล์จาก host (data/hello.txt มีอยู่แล้วใน repo)
docker compose exec app cat /app/data/hello.txt
# → Hello from host!

# container เขียนไฟล์ใหม่ — ปรากฏบน host ทันที
docker compose exec app sh -c 'echo "Written by container" >
/app/data/from-container.txt'
cat data/from-container.txt
# → Written by container

ls data/
# hello.txt  from-container.txt

docker compose down
cd ../../

```

### ขั้นตอนที่ 8 ทดสอบ Named Volume Share

เปิดดู volume-shared/docker-compose.yml — writer และ reader ใช้ shared-data:/data volume เดียวกัน

```
cd lab2b-examples/volume-shared
cat docker-compose.yml          # ศึกษา compose file
docker compose up -d

# ดู log ของ reader — ควรเห็นข้อมูลที่ writer เขียน
docker compose logs reader
# → Hello from writer!
# → Line 2

# ตรวจสอบ named volume
docker volume ls
docker volume inspect volume-shared_shared-data

docker compose down -v          # -v ลบ named volume ด้วย
cd ../../
```